# Adaptive KV Materialization Control for Multi-GPU LLM Serving

Anonymous Artifact Draft

Abstract—Large language model serving stacks increasingly expose reusable prefix or KV state across requests, but converting that state into startup-latency reduction is not free. Realizing reusable state may require transfer, stitching, or format adaptation, so a runtime needs a decision at request arrival: should it fully reuse that state, partially materialize only a profitable segment, or recompute from scratch? This artifact studies that narrow three-action control surface. Its workload-driven offline path evaluates the arrival-time decision surface over named shared cases from llm-serving-workloads; its live path documents what the current out-of-tree vLLM seam actually realizes online. The resulting claim boundary is explicit: the repository supports decision-side evidence for full_reuse, partial_reuse, and recompute, and live runtime evidence for native full_reuse/recompute plus truthfulness-safe partial_reuse fallback reporting on the current prefix-cache path.

Index Terms—LLM serving, KV cache, prefix reuse, materialization control, time to first token

## I. Introduction

Recent LLM serving systems increasingly rely on prefix reuse, prompt caching, and reusable KV state to reduce prefill cost. However, a reusable prefix is not equivalent to a free latency reduction. In multi-GPU settings, reusable state may reside on another worker, may require transfer through interconnect or host memory, and may need partial rematerialization before decoding can start. As a result, the serving system faces a control problem: when a request arrives and some reusable state exists, how much of that state should be realized now?

Recent disaggregated and remote-KV serving efforts, including DistServe and Mooncake [1], [2], make this cost visible rather than hiding it behind a monolithic "cache hit" abstraction. Once reuse requires transport, re-layout, or partial reconstruction, the runtime needs a policy that decides whether realizing the reusable state is worth the startup cost.

This repository focuses on a narrow but practically important control point: request-arrival KV materialization. Instead of treating reuse as a binary choice, we study a three-action decision surface. The controller can fully materialize the reusable prefix, partially materialize only a profitable segment, or ignore the reusable state and recompute the prompt from scratch. This formulation is intentionally orthogonal to placement, admission, and online memory-evolution policy. Placement affects where reusable state resides; admission affects which work enters the system; memory evolution affects how state is written, retained, or retired over time. Our question begins after reusable state is already available and asks whether realizing it at request arrival is worth the cost.

The systems hypothesis is straightforward: the best action depends on the tradeoff between recomputation work and realization overhead. Realization overhead can include transfer latency, bytes moved, device memory pressure, and stitching cost. Recompute cost depends on prompt length, model throughput, and request urgency. Because these factors vary across workloads, a serving stack needs an explicit materialization controller instead of a fixed "always reuse" rule.

In contrast to a broad scheduler redesign, this paper targets one narrow and measurable hook. The benefit of that choice is methodological: it keeps the artifact honest about what is being optimized, and it makes offline and online evidence easier to interpret.

## II. Problem Formulation

We study a single-hook decision problem at request arrival. For each incoming request, the runtime observes a candidate reusable prefix together with signals about prefix length, estimated recomputation work, and estimated realization cost. The controller then chooses one of three actions:

- full_reuse: materialize the full reusable prefix,
- partial_reuse: materialize only a prefix segment and recompute the rest,
- recompute: ignore reusable state and run the prompt path from scratch.

This decision surface is deliberately narrow. It avoids conflating multiple control layers and makes the evidence interpretable: if the controller helps, the gain comes from better request-arrival materialization decisions rather than from a hidden change in scheduler, placement, or eviction behavior.

### A. Signals and Objective

The controller reasons over a small signal set: reusable prefix length, estimated transfer latency, estimated recomputation latency, queue pressure, and a confidence estimate for whether the candidate reusable state is likely to produce useful saved work. The objective is to minimize startup latency, represented primarily by TTFT, while keeping realized KV traffic proportional to actual benefit.

### B. Why Partial Reuse Matters

The partial action is the key difference between a binary reuse controller and a materialization controller. In practice, the early segment of a reusable prefix may be worth loading while the tail segment is not, especially when transfer cost grows faster than the remaining recomputation avoided. A binary policy cannot express that breakpoint.

### C. Current Partial-Reuse Boundary

The artifact keeps the boundary around partial_reuse explicit. In the offline decision study, partial_reuse is a first-class action: the controller may materialize only a profitable prefix segment and recompute the remainder. On the current live prefix-cache seam, exact online segmented materialization is not yet available. Accordingly, the runtime does not claim native support it does not have: it logs partial_reuse as the observed policy decision, marks the support tier as a fallback, records the reason that exact partial segment materialization is unavailable on the current prefix-cache path, and degrades the effective online action to anchor-scoped full_reuse.

## III. Design Perspective

The design goal is not to maximize reuse rate. The design goal is to minimize request latency subject to the actual cost of realizing reusable state. From this perspective, full reuse is only one candidate action, not the default answer. A partial action is useful when the early portion of a prefix has high reuse value but the tail has poor return once transfer and stitching overhead are considered. The recompute action remains necessary when realization cost dominates the saved prefill work.

This framing is important for systems evaluation. A policy that increases nominal reuse coverage but also increases transfer volume or startup delay is not an optimization. We therefore evaluate the controller with both latency and cost-oriented metrics, rather than reuse ratio alone.

This design perspective also sets the paper's evidence boundary. A strong offline result means the decision rule is plausible under workload-grounded cost assumptions. It does not automatically imply an end-to-end serving gain until the same rule is tested through the live vLLM plugin path. Conversely, a live run that only observes decisions without enforcing them is useful for validating workload entry, prompt geometry, and service-side constraints, but it is not yet evidence that the controller changed runtime behavior.

## IV. Methodology

This artifact uses two evaluation layers and keeps them separate.

The first layer is a workload-driven offline decision study over named shared cases from llm-serving-workloads. This layer is used to compare the three-action decision surface across baselines, heuristic policies, oracle headroom, and cost-misestimation sensitivity variants.

The second layer is a live runtime-boundary path against a controlled Qwen2.5-7B-Instruct deployment. This layer is used to report which actions the current runtime seam realizes online and which actions still require fallback.

We compare fixed baselines that always recompute, always fully reuse, or apply a simple thresholded partial policy. We then evaluate an adaptive heuristic controller that estimates whether realization cost is justified. To bound the available policy headroom, we also report an oracle selector over the same three-action space. Finally, we include cost-misestimation sensitivity variants to test whether the heuristic remains stable when the runtime's cost model is inaccurate.

The primary metrics are mean and tail TTFT, recomputed prompt tokens, and KV bytes realized or transferred. Together these metrics distinguish beneficial reuse from expensive reuse.

### A. Workload Source of Truth

The study deliberately enters through the sibling llm-serving-workloads repository rather than a repo-local toy catalog. This keeps workload construction policy-neutral and prevents the paper from baking a one-off benchmark family directly into the optimization artifact.

### B. Baselines

The baseline set is intentionally simple and interpretable: always recompute, always full reuse, and a fixed threshold-based partial policy. The adaptive heuristic is then compared against these baselines, while an oracle selector is used only to estimate remaining headroom within the same three-action space.

### C. Current Evidence Scope

The repository currently supports a six-case offline decision matrix covering exact continuation, long-context retrieval follow-up, structured transcript overlap, prefix-rich multi-tenancy, long-context continuation, and dynamic retrieval follow-up. It also supports a live runtime-boundary path that enters through the same shared workload repository and runs against a controlled Qwen2.5-7B-Instruct endpoint.

The claim boundary is therefore explicit. The offline layer supports claims about the three-action arrival-time decision surface. The live layer supports claims about runtime realization status on the current seam: native support for full_reuse and recompute, and fallback reporting for partial_reuse.

## V. Live Runtime Boundary

The live path is intentionally narrower than a broad online benchmark suite. It is used to answer one question: what does the current runtime seam really do?

On the current out-of-tree prefix-cache path, full_reuse is realized through stable anchor-scoped cache domains, and recompute is realized through request-scoped cache bypass. partial_reuse is not yet realized as exact segmented materialization and must therefore be reported as a fallback on this path.

### A. Fallback Taxonomy

This distinction is essential for paper truthfulness. The repository has a real online control path for two of the three actions, but it does not yet have an end-to-end implementation of exact online partial_reuse. The runtime therefore exposes a fallback taxonomy rather than hiding the gap. When the policy selects partial_reuse, the live record includes the observed decision, the effective fallback action, the support tier, and the fallback reason stating that exact partial segment materialization is unavailable on the current prefix-cache path.

## VI. Offline Decision Study

The offline study remains valuable because it is the only place where the full three-action decision surface is currently evaluated uniformly across all selected workload cases. It is the right layer for reasoning about whether partial_reuse has a meaningful decision region and how that region changes across workload families.

## VII. Artifact Boundary

This repository intentionally separates decision-side evidence from live runtime realization evidence. We do not present offline oracle results as deployed gains, and we do not treat the existence of reusable state as proof that materialization is beneficial. We also do not reframe this repository as a general admission or memory-evolution artifact simply because some workloads contain broader state semantics.

## VIII. Current Contributions

1) A precise systems formulation of request-arrival KV materialization as a three-action control problem.
2) A workload-driven six-case decision matrix sourced entirely from llm-serving-workloads, including prefix-rich, long-context continuation, and dynamic retrieval-follow-up surfaces.
3) An explicit evidence boundary separating offline decision-surface claims from live runtime-boundary claims.
4) A narrow out-of-tree vLLM plugin target centered on materialization control rather than scheduler, admission, or memory-evolution redesign, with a live fallback taxonomy that truthfully reports the current partial_reuse gap.

## IX. Limitations and Next Step

The main limitation remains exact online partial_reuse. The next milestones are therefore concrete:

1) implement a deeper runtime path, likely via a custom KV materialization or connector seam, that can realize exact online partial_reuse rather than degrading it to anchor-scoped full reuse;
2) keep the live runtime-boundary matrix grounded in shared workload cases that stress prefix-rich, long-context continuation, and dynamic retrieval follow-up behavior; and
3) rerun the shared workload suite under the deeper runtime path once exact online segmented materialization exists.

## References

[1] "Distserve," 2024.
[2] "Mooncake," 2025.